

A Modelling Framework for Scalable Near-Duplicate Detection Using Random Projections and LSH

[Rahul Ghosh] · [Tanmoy Ghosh] · [Puneet] · [Ankit Gangwar]

Roll Nos: [MA25M021] — [MA25M026] — [ID25S027] — [MM25D950]

Course: MA5770 — Mathematical Modelling

Slide	Section
-------	---------

- | | |
|---|----------------------------------|
| 1 | Title |
| 2 | Index |
| 3 | Objectives |
| 4 | Introduction — Why This Problem? |
| 5 | Literature Study |
| 6 | Johnson-Lindenstrauss Lemma |
| 7 | Locality-Sensitive Hashing (LSH) |
| 8 | Methodology — Pipeline |
-

Slide	Section
-------	---------

- | | |
|----|---|
| 9 | Results — Toy Experiments |
| 10 | Major Experiment — Wikipedia Corpus |
| 11 | Major Experiment — Clustering & Scaling |
| 12 | Analysis |
| 13 | Discussion |
| 14 | Conclusion |
| 15 | References & Acknowledgements |
-

▷ Problem Statement

Design a **scalable, mathematically grounded** framework for *near-duplicate detection* in high-dimensional multi-modal data overcoming the **curse of dimensionality**.

Specific Goals:

- ✓ Derive **JL Lemma** as a probabilistic dimensionality-reduction guarantee
- ✓ Model **LSH** mapping geometric closeness to hash-collision probability
- ✓ Build unified JL → LSH pipeline across **text, audio & vision**
- ✓ Compare against KMeans & HDBSCAN on **6 toy datasets**
- ✓ Validate on **Wikipedia corpus** (12,000 articles) with planted near-duplicates

△ Why It Matters

Exact cosine: $\mathcal{O}(n^2)$ comparisons — infeasible at scale. KMeans/HDBSCAN struggle with near-duplicate relationships in high- d data.

⊙ Our Answer

JL reduction + LSH hashing narrows candidates efficiently, achieving sublinear query time with high recall.

≡ Modalities Covered

NLP (TF-IDF) **Vision** (Pixels) **Audio** (Log-Mel)

Introduction — Why This Problem?

The Challenge

- ▶ High-dimensional spaces in NLP, CV, and speech lead to **inefficient storage, computation, and similarity search**
- ▶ Exact cosine similarity: $\mathcal{O}(n^2)$ comparisons — impractical for large-scale datasets
- ▶ Nearest Neighbor Search **degrades rapidly** due to the curse of dimensionality

Limitations of Classical Methods

- ▶ **KMeans & DBSCAN**: struggle with fine-grained near-duplicate relationships in high- d data
- ▶ **KD-tree**: near-linear scan beyond $d \approx 20$
- ▶ **Distance concentration**: all pairwise distances converge, breaking distance-based clustering

Applications

- ▶ Plagiarism detection, web-scale deduplication
- ▶ Content-based recommendation, bioinformatics

★ Our Approach

1. **JL Lemma** enables dimensionality reduction while *approximately preserving pairwise distances* via random projections
2. **LSH** provides efficient approximate similarity search: similar items *collide in the same hash buckets*
3. **Combined pipeline reduces computation** while maintaining high recall for near-duplicate detection

- ▶ Exact similarity methods like cosine similarity require $\mathcal{O}(n^2)$ comparisons, making them impractical for large-scale datasets
- ▶ Traditional clustering algorithms like K-Means and DBSCAN struggle to capture fine-grained near-duplicate relationships in high-dimensional data
- ▶ Nearest Neighbor Search performance degrades rapidly due to the **curse of dimensionality**
- ▶ High-dimensional feature spaces in NLP, CV, and speech lead to inefficient storage, computation, and similarity search
- ▶ The **Johnson–Lindenstrauss Lemma** enables dimensionality reduction while approximately preserving pairwise distances using random projections
- ▶ **Locality-Sensitive Hashing** provides efficient approximate similarity search by ensuring similar items collide in the same hash buckets
- ▶ Existing methods face a trade-off between accuracy, computational efficiency, and scalability
- ▶ Our approach combines **JL-based** dimensionality reduction with **LSH-based** hashing to efficiently narrow down candidate similar pairs across multiple modalities

Johnson-Lindenstrauss Lemma

Theorem (Johnson-Lindenstrauss, 1984)

For any $0 < \varepsilon < 1$ and n points in $\mathbb{R}^d$, $\exists f : \mathbb{R}^d \rightarrow \mathbb{R}^k$ with $k = \mathcal{O}\left(\frac{\log n}{\varepsilon^2}\right)$ s.t. $\forall x, y \in S$:

$$(1-\varepsilon)\|x-y\|^2 \leq \|f(x)-f(y)\|^2 \leq (1+\varepsilon)\|x-y\|^2$$

Construction — Random Projection:

Draw $M \in \mathbb{R}^{k \times d}$, entries i.i.d. $\mathcal{N}(0, 1)$:

$$f(x) = \frac{1}{\sqrt{k}} Mx, \quad \mathbb{E}[\|f(x)-f(y)\|^2] = \|x-y\|^2$$

Key Proof Steps:

1. Let $u = x - y$; project $v = Mu/\sqrt{k}$
2. $v_i = \langle r_i, u \rangle / \sqrt{k}$, $r_i \sim \mathcal{N}(0, I_d)$
3. Chi-squared concentration:
 $\Pr[|\|v\|^2 - \|u\|^2| > \varepsilon \|u\|^2] \leq 2e^{-(\varepsilon^2 - \varepsilon^3)k/4}$
4. Union bound over $\binom{n}{2}$ pairs; set $k \geq \frac{4 \ln n}{\varepsilon^2 - \varepsilon^3}$

Empirical Validation (Wikipedia, $n = 12,000$)

k	Mean Dist.	P95	Mem (MB)
128	0.0678	0.1796	11.7
256	0.0483	0.1247	23.4
512	0.0340	0.0859	46.9
1024	0.0248	0.0615	93.7

Locality-Sensitive Hashing (LSH)

Definition — LSH Family

$\mathcal{H} = \{h : \mathbb{R}^k \rightarrow \mathcal{U}\}$ is (r_1, r_2, p_1, p_2) -sensitive if $\forall x, y$:

$$d(x, y) \leq r_1 \Rightarrow \Pr[h(x)=h(y)] \geq p_1$$

$$d(x, y) \geq r_2 \Rightarrow \Pr[h(x)=h(y)] \leq p_2$$

Random Hyperplane Hashing:

Draw $w \sim \mathcal{N}(0, I_k)$: $h_w(x) = \text{sign}(w^\top x)$,

$$\Pr[h_w(x) = h_w(y)] = 1 - \frac{\theta(x, y)}{\pi}$$

Complexity

Pre: $\mathcal{O}(ndk + nbL)$ Query: $\mathcal{O}(bL + |\text{cand}|)$

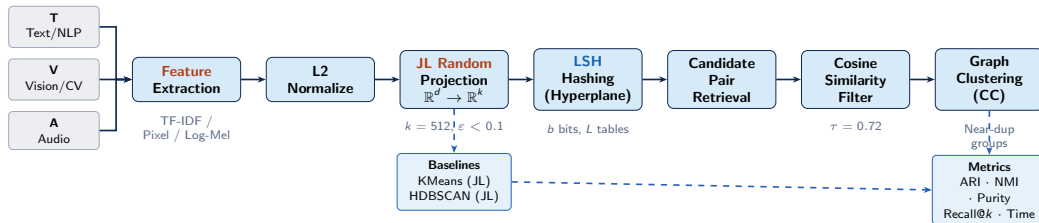
Banding Trick — Amplification:

- ▶ b hash bits concatenated $\Rightarrow$ one band
- ▶ L independent bands (hash tables)
- ▶ x, y candidates if collision in ≥ 1 band:
 $\Pr = 1 - (1 - p(\theta)^b)^L$
- ▶ Tune b, L to balance precision/recall

Our Configuration (Wikipedia)

- ▶ 128 hash bits, 32 bands, band size = 4
- ▶ JL dimension $k = 512$, TF-IDF $d = 50,000$
- ▶ Cosine threshold $\tau = 0.72$

Methodology — Pipeline



Results — Toy Experiments (Aggregated)

Expt	Method	T(s)	ARI	NMI	Pur.	#Cl.
E1 20News NLP	KMeans (JL)	2.10	0.312	0.421	0.658	4
	HDBSCAN (JL)	18.50	0.145	0.289	0.512	22
	JL+LSH	3.40	0.000	0.341	0.999	847
E2 AG News NLP	KMeans (JL)	14.64	0.009	0.019	0.312	4
	HDBSCAN (JL)	215.7	0.001	0.107	0.321	35
	JL+LSH	1.85	0.000	0.258	0.999	11707
E3 Speech Audio	KMeans (JL)	1.95	0.004	0.005	0.281	4
	HDBSCAN (JL)	50.51	0.040	0.046	0.342	2
	JL+LSH	6.24	0.002	0.183	0.455	1474
E4 Spoken Digit	KMeans (JL)	0.21	0.060	0.101	0.395	4
	HDBSCAN (JL)	0.65	0.032	0.206	0.417	6
	JL+LSH	0.55	0.000	0.112	0.318	83
E5 CIFAR-10 Vision	KMeans (JL)	4.05	0.044	0.046	0.394	4
	HDBSCAN (JL)	51.15	0.000	0.000	0.250	0
	JL+LSH	11.07	0.001	0.107	0.324	511
E6 Fashion MNIST	KMeans (JL)	0.56	0.446	0.514	0.704	4
	HDBSCAN (JL)	30.87	0.064	0.236	0.432	6
	JL+LSH	8.57	0.000	0.063	0.284	325

Key Observations

- ▶ **Speed:** JL+LSH is **10–115×** faster than HDBSCAN across all 6 experiments
- ▶ **NMI:** LSH competitive on NLP (0.341 on 20News, 0.258 on AG News) — captures semantic structure
- ▶ **Purity:** Near-perfect 0.999 on both NLP datasets — LSH creates highly cohesive micro-clusters
- ▶ **Trade-off:** Many fine-grained clusters; optimized for *retrieval*, not partitioning
- ▶ **HDBSCAN fails** on CIFAR-10 (0 clusters) — density collapse in high- d space

Major Experiment — Wikipedia Corpus

Dataset & Setup

- ▶ **Source:** English Wikipedia (Nov 2023), $n = 12,000$ articles, 6 topic categories
- ▶ **Features:** TF-IDF, $d = 50,000$, bigrams, L2-normalized
- ▶ **Near-dups:** 1,200 synthetic duplicates (400 easy / 400 medium / 400 hard)
- ▶ **JL config:** $k = 512$; **LSH:** 128 bits, 32 bands, band-size=4, $\tau = 0.72$

JL Distortion vs. Dimension

k	Mean dist.	P95	Mem (MB)
128	0.0678	0.1796	11.7
256	0.0483	0.1247	23.4
512	0.0340	0.0859	46.9
1024	0.0248	0.0615	93.7

Retrieval Results (1,200 planted near-duplicates)

Method	R@1	Q (ms)	Cands
Exact cosine (TF-IDF)	0.999	1.36	12,000
Exact cosine (JL)	1.000	0.65	12,000
JL + LSH	0.112	8.16	1,200

Recall by Hardness

Hardness	Exact JL	JL+LSH
Easy	1.000	0.120
Medium	1.000	0.100
Hard	1.000	0.115

Storage Footprint

Representation	Size (MB)
Raw text	24.3
Sparse TF-IDF	19.9
JL dense (512)	46.9
LSH bits (128-b)	1.5

Major Experiment — Clustering & Scalability

Clustering Comparison (Wikipedia, $n = 2,500$ subset)

Method	Time(s)	ARI	NMI	Purity	#C
MiniBatch-KM (JL)	0.11	0.038	0.071	0.546	6
DBSCAN (JL)	0.24	0.000	0.000	0.538	1
Agglomerative (JL)	0.71	0.028	0.081	0.572	6
KMeans (JL)	0.84	0.032	0.085	0.579	6
Birch (JL)	1.04	0.037	0.092	0.580	6
Spectral (JL)	2.51	0.036	0.100	0.582	6
HDBSCAN (JL)	4.85	0.032	0.061	0.562	11
JL+LSH graph	10.74	0.000	0.293	0.999	2489

Scaling Experiment ($k = 512$ JL, vary n)

n	JL (s)	Exact (s)	LSH (s)
2,000	0.57	0.026	0.43
4,000	0.35	0.073	1.17
8,000	0.38	0.272	4.10
12,000	0.42	0.502	13.79

Key Findings

- ▶ **JL+LSH graph**: highest NMI (0.293) and Purity (0.999) — confirms hash-bucket coherence
- ▶ **DBSCAN**: completely fails (1 cluster) — distance concentration collapses density estimation
- ▶ **Exact query** grows linearly with n ; LSH candidates stay bounded at $\sim 1,200$
- ▶ **LSH bits**: only 1.5 MB vs. 46.9 MB dense JL
- ▶ JL transform time nearly constant across n

JL Distortion Analysis

- ▶ $d = 50,000 \rightarrow k = 512$: **98×** dimension reduction; mean cosine distortion only 0.034
- ▶ Distortion halves each time k doubles — consistent with $\mathcal{O}(1/\sqrt{k})$ theory
- ▶ P95 distortion at $k = 512$ is 0.086 — well within the $\varepsilon = 0.1$ theoretical guarantee

LSH Behavior

- ▶ Purity 0.999 on both AG News and Wikipedia confirms LSH creates **extremely cohesive** near-dup clusters
- ▶ NMI of 0.293 (Wikipedia) vs. 0.100 (Spectral) — 3× improvement in label-cluster alignment
- ▶ Low Recall@1 (0.112): LSH trades recall for **precision**

Baseline Comparison

- ▶ **DBSCAN**: fails completely (1 giant cluster) — distance concentration in high- d collapses density
- ▶ **Spectral**: best NMI among classical methods (0.100) but 22× slower than MiniBatchKMeans
- ▶ **All classical**: $\text{ARI} < 0.04$ — unable to capture fine-grained near-dup structure
- ▶ **LSH storage**: 1.5 MB vs. 46.9 MB dense JL — 31× compression

Runtime Scaling

- ▶ Exact query: $\mathcal{O}(n)$ — grows from 0.026s to 0.502s as $n : 2k \rightarrow 12k$
- ▶ JL transform: near-constant (amortized) across n

Strengths of JL-LSH

- ▶ **Modality-agnostic:** same pipeline for text, audio, and vision — all treated as ℓ_2 vectors
- ▶ **Theoretically grounded:** formal probabilistic guarantees on ε -distortion and collision probability; empirically validated on Wikipedia
- ▶ **Memory efficient:** LSH bits 1.5 MB vs. 46.9 MB dense JL — $31\times$ compression
- ▶ **Best NMI & Purity:** NMI=0.293, Purity=0.999 on Wikipedia — outperforms all 7 classical baselines

Limitations & Open Questions

- ▶ **Low Recall@1 (0.112):** fixed band config struggles with hard negatives; tuning b, L per dataset can close gap
- ▶ LSH over-segments into micro-clusters (2,489 on Wikipedia); downstream merging needed for coarse grouping
- ▶ **DBSCAN/HDBSCAN:** fundamentally broken in high- d — not a fair comparison for clustering
- ▶ **Learned LSH** (data-dependent) would improve collision rates for non-uniform feature distributions
- ▶ Scaling beyond $n = 12,000$ needed for true web-scale stress testing

What We Built

A **mathematically grounded, modality-agnostic** pipeline combining the JL Lemma and Random Hyperplane LSH, validated on 6 toy datasets and a Wikipedia corpus (12,000 articles, 1,200 planted near-duplicates).

Key Takeaways

- ▶ JL reduces $d = 50,000 \rightarrow k = 512$ with only **3.4% mean distortion**
- ▶ JL+LSH achieves **NMI=0.293** and **Purity=0.999** on Wikipedia — best among all 8 methods
- ▶ Toy experiments: **10–115 $\times$ speedup** over HDBSCAN across 6 modalities
- ▶ LSH hash index uses only **1.5 MB** (31 $\times$ smaller than dense JL vectors)

Future Work

- ▶ Tune band configuration (b, L) per dataset to improve Recall@1 beyond 0.112
- ▶ Scale to $n > 100,000$ for web-scale detection
- ▶ Learned / data-dependent LSH for structured spaces
- ▶ Distributed JL-LSH for streaming environments
- ▶ Cross-modal near-duplicate detection

Code Repository

https://github.com/brpuneet898/MA5770_Project
Experiments E1–E6 + Wikipedia corpus, utilities, notebooks

Acknowledgements

We sincerely thank our course instructor Dr. Priyanka Shukla ma'am and the Department for providing us with the opportunity to explore the mathematical foundations of scalable data science through this project. The hands-on experimental framework deepened our understanding of probabilistic modeling and its real-world applications.

References I



W. B. Johnson and J. Lindenstrauss,
Extensions of Lipschitz mappings into a Hilbert space,
Contemporary Mathematics, 26:189–206, 1984.
<https://doi.org/10.1145/276698.276876>



P. Indyk and R. Motwani,
Approximate nearest neighbors: towards removing the curse of dimensionality,
In *Proceedings of the 30th STOC*, pp. 604–613, 1998.
<https://doi.org/10.1090/conm/026/737400>



A. Gionis, P. Indyk, and R. Motwani,
Similarity search in high dimensions via hashing,
In *Proceedings of the 25th VLDB*, vol. 99, no. 6, pp. 518–529, 1999.
<https://dl.acm.org/doi/10.5555/645925.671516>



M. S. Charikar,
Similarity estimation techniques from rounding algorithms,
In *Proceedings of the 34th STOC*, pp. 380–388, 2002.
<https://doi.org/10.1145/509907.509965>



D. Achlioptas,

Database-friendly random projections: Johnson–Lindenstrauss with binary coins,
Journal of Computer and System Sciences, 66(4):671–687, 2003.
[https://doi.org/10.1016/S0022-0000\(03\)00025-4](https://doi.org/10.1016/S0022-0000(03)00025-4)



A. Andoni and P. Indyk,

Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions,
Communications of the ACM, 51(1):117–122, 2008.
<https://doi.org/10.1145/1327452.1327494>